



viettel

Theo cách của bạn



VIETTEL DIGITAL TALENT 2026

Streaming Processing in BigData Platform

Real-time data pipelines with Apache Kafka & Apache Flink

Mentor: Nguyen Minh Hieu

Student: Ngo Quang Hieu

Data Engineer Track
Viettel Digital Talent 2026

July 1st, 2026

Table of Contents

- 1 Introduction
- 2 Project Architecture
- 3 Message Queue: Apache Kafka
- 4 Stream Processing: Apache Flink
- 5 Data Warehouse & Dashboard
- 6 Monitoring
- 7 Conclusion

Table of Contents

- 1 Introduction**
- 2 Project Architecture
- 3 Message Queue: Apache Kafka
- 4 Stream Processing: Apache Flink
- 5 Data Warehouse & Dashboard
- 6 Monitoring
- 7 Conclusion

Why Stream Processing?

The Problem with Batch

- **High latency** — jobs run hourly/daily, insights lag by hours
- **Stale decisions** — by the time data is ready, the moment to act has passed
- **Resource spikes** — large periodic loads strain infra, then sit idle

Batch: hours of delay



Stream: milliseconds

Stream Processing Enables

- **Live monitoring & alerting** — detect anomalies as they happen
- **Continuous aggregation** — dashboards show current metrics, not a stale snapshot
- **Smooth, steady load** — events processed as they arrive, no periodic resource spikes

Objectives

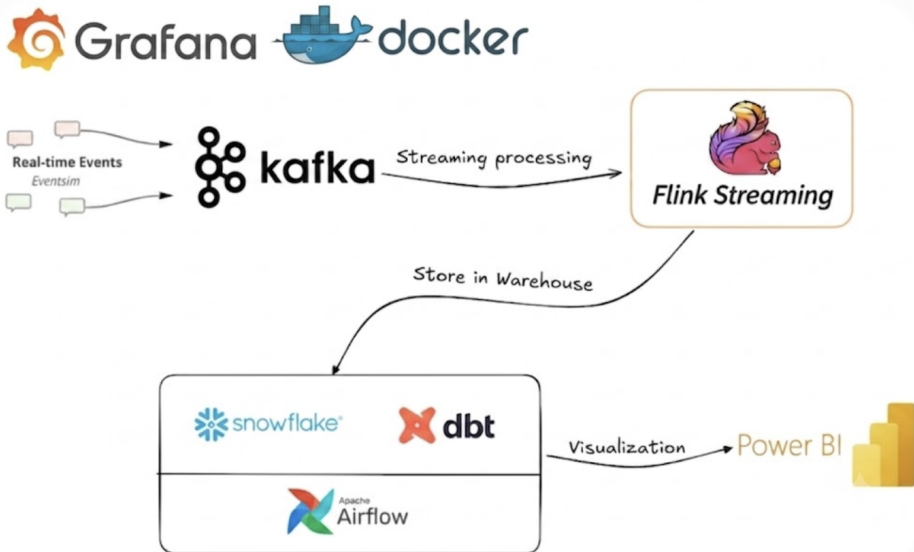
Objectives

- ➊ Overview **Apache Kafka** as a distributed message queue
- ➋ Overview **Apache Flink** as a real-time stream processing engine
- ➌ Build an end-to-end **ELT streaming pipeline** — from event simulation to BI dashboard
- ➍ Demonstrate **observability** with Prometheus & Grafana monitoring

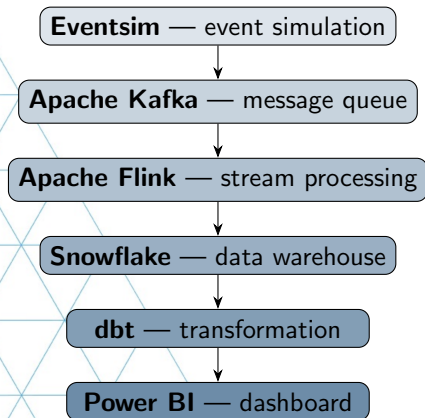
Table of Contents

- 1 Introduction
- 2 Project Architecture**
- 3 Message Queue: Apache Kafka
- 4 Stream Processing: Apache Flink
- 5 Data Warehouse & Dashboard
- 6 Monitoring
- 7 Conclusion

Streamify: End-to-End Architecture



Architecture: Layer by Layer



3 Event Streams Simulated

- `listen_events` — song plays, skips, durations
- `page_view_events` — user navigation behaviour
- `auth_events` — logins and session events

Deployment

- Fully containerised with **Docker Compose**
- Single command: `docker compose up`
- Flink job auto-submitted on startup

Table of Contents

- 1 Introduction
- 2 Project Architecture
- 3 Message Queue: Apache Kafka**
- 4 Stream Processing: Apache Flink
- 5 Data Warehouse & Dashboard
- 6 Monitoring
- 7 Conclusion

Why Apache Kafka?

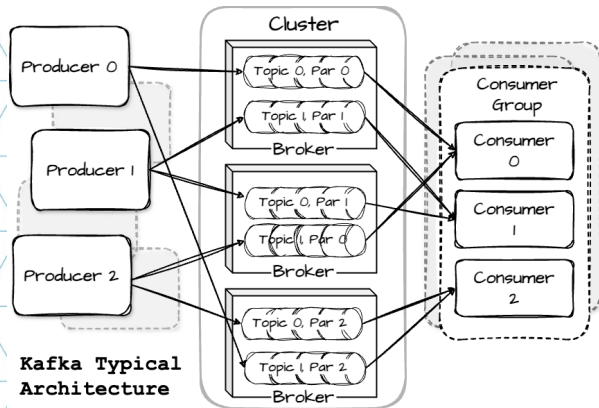
Problems Without a Message Queue

- **Tight coupling** — producer knows every consumer
- **No buffering** — slow consumer → data loss
- **No replay** — consumed = gone forever
- **Fan-out cost** — N consumers = N direct calls

How Kafka Solves It

- **Decoupling** — write to topic, subscribe independently
- **Durable buffer** — persisted to disk, replicated
- **Replay** — configurable retention, re-read anytime
- **Scalability** — partitions scale throughput linearly
- **Fan-out free** — multiple groups, own offsets

Kafka Architecture



- **Producer** — partitions by key; acks=all for durability
- **Broker** — leader/follower replicas; append-only log
- **Topic** — immutable records; retention-based or compacted
- **Partition** — ordered, offset-indexed; unit of parallelism
- **Consumer Group** — one partition → one member; independent offset per group

Max parallelism = number of partitions

Kafka Setup in Streamify

3-Node KRaft Cluster

- Runs on **apache/kafka:4.3.0** — no Zookeeper
- Each node is both broker & controller
- **Replication factor 3**, min ISR 2 — no data loss if one broker restarts

Topic Configuration

- 3 topics: `listen_events`, `page_view_events`, `auth_events`
- **3 partitions** per topic — enables parallel consumption by Flink
- Topics pre-created by `kafka-init` before producers start

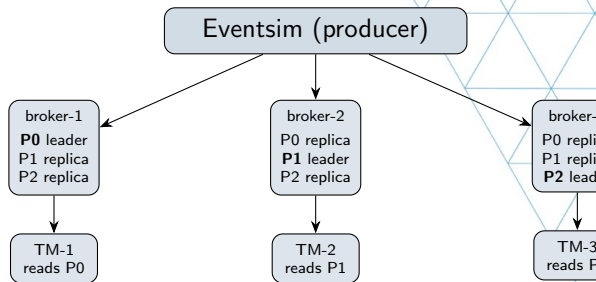
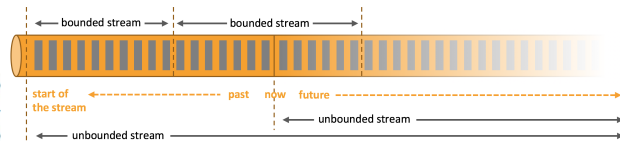


Table of Contents

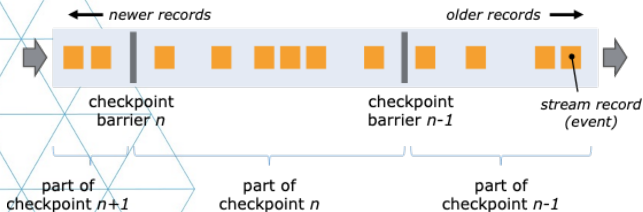
- 1 Introduction
- 2 Project Architecture
- 3 Message Queue: Apache Kafka
- 4 Stream Processing: Apache Flink**
- 5 Data Warehouse & Dashboard
- 6 Monitoring
- 7 Conclusion

Apache Flink: True Event Streaming



True streaming: records flow through operator DAG one-by-one

data stream



Chandy-Lamport barriers: async snapshots without pausing processing

Dataflow Model

- **Record-at-a-time** — no batching delay
- Operators form a DAG; data flows continuously
- **Event time + watermarks** — correct out-of-order handling

Fault Tolerance

- Barrier checkpoints injected into the stream
- **Exactly-once** with transactional sinks
- State stored in heap or RocksDB
- Fully async — no pause in processing

Flink Setup in Streamify

Scaled Deployment

- **1 JobManager** — coordinates job scheduling and checkpoints
- **3 TaskManagers** — each handles one partition of each topic in parallel
- **Parallelism 3** — matches 3 Kafka partitions exactly
- Slot sharing: one full pipeline per TaskManager

PyFlink Pipeline

- Source: Kafka SQL connector (3 topics)
- Transform: Python UDF — JSON decode, timestamp enrichment
- Sink: Snowflake via JDBC (exactly-once)
- Checkpointing every **10 seconds** — commits offsets back to Kafka

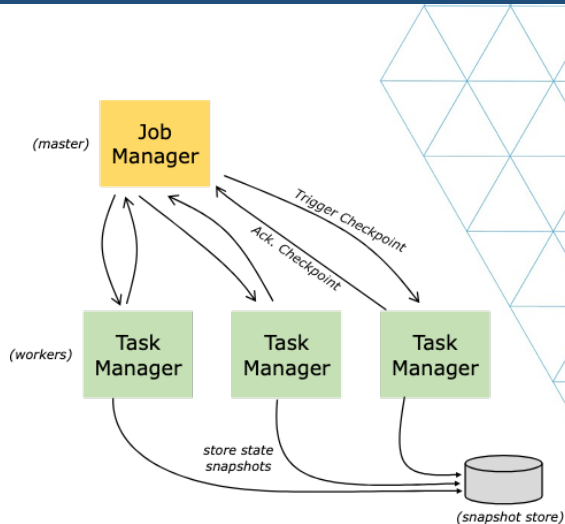
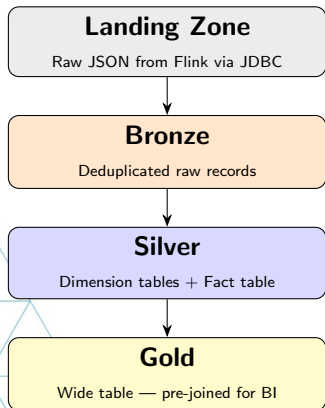


Table of Contents

- 1 Introduction
- 2 Project Architecture
- 3 Message Queue: Apache Kafka
- 4 Stream Processing: Apache Flink
- 5 Data Warehouse & Dashboard**
- 6 Monitoring
- 7 Conclusion

Warehouse: Landing Zone → Gold



Landing Zone

- Raw events written directly by Flink — no transformation

Bronze — Deduplication

- Remove duplicate events using event ID and timestamp

Silver — Star Schema

- `fact_streams` central fact table
- 5 dimension tables: users, songs, artists, locations, datetime

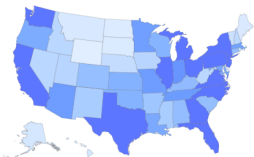
Gold — Wide Table

- `wide_streams`: pre-joins fact + all dims
- Single flat table for Power BI — no runtime JOINS

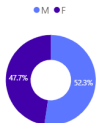
Dashboard: Real-time Insights

STREAMIFY

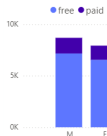
User Activity per State



Gender Distribution



User Level By Gender



User Activity



State Filter
All

Streams
327K

Total Users
16.39K

Date Range
8/20/2025 8/27/2025

Chart Busters

Song	Streams
Yo Yo Man (Garter Belt)	1,352
Evenglow	812
Up Jumped The Devil - Live	778
Una dolce melodia	754
Close Your Eyes	753
Wish (Album Version)	642
Alfil_ Ella No Cambia Nada	588
Intro	447
So Damn High (Will Eastman Club Edit)	398
Inferior (Late Night Version)	312

Top Artists

Artist	Streams
Tony Joe White	1,585
David & Steve Gordon	906
Atomic Rooster	861
Clarence Gamemouth Brown	815
Franco_ Valeriana	754
Luis Alberto Spinetta	699
Beautiful Creatures	642
Phil Collins	556
Sugar Minott	521
The Jackson Southemaires	485

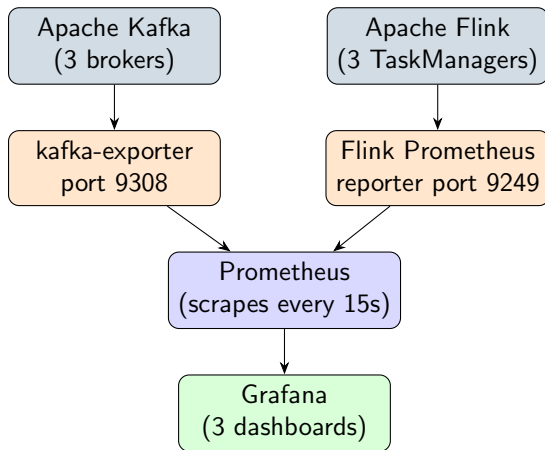
Key Metrics

- **Top artists** by total play count
- **Songs played over time** — hourly trend
- **Geographic distribution** — listener locations (US map)
- **Free vs. paid** listener split
- **Session activity** — active users over time

Table of Contents

- 1 Introduction
- 2 Project Architecture
- 3 Message Queue: Apache Kafka
- 4 Stream Processing: Apache Flink
- 5 Data Warehouse & Dashboard
- 6 Monitoring**
- 7 Conclusion

Monitoring: How It Fits Together



Monitoring Stack: Prometheus & Grafana

Why Monitor?

- Verify Flink throughput matches Kafka ingestion rate
- Detect consumer-group lag
- Track CPU and memory anomalies across all 3 TaskManagers and the JobManager

Components

- **Flink Prometheus Reporter** — exposes JVM and operator metrics on port 9249
- **kafka-exporter** — exposes consumer-group lag on port 9308
- **Prometheus** — scrapes both; stores time-series data
- **Grafana** — visualises with 3 pre-built dashboards

3 Grafana Dashboards

Kafka Lag

Consumer-group lag per topic + consumption rate (msgs/s) — detects if Flink is falling behind

Flink Record Processing

Records ingested per second per source operator — confirms data is flowing through the pipeline

Flink CPU / Memory

Per-TaskManager JVM CPU load, heap usage, and managed memory — identifies resource pressure

Table of Contents

- 1 Introduction
- 2 Project Architecture
- 3 Message Queue: Apache Kafka
- 4 Stream Processing: Apache Flink
- 5 Data Warehouse & Dashboard
- 6 Monitoring
- 7 Conclusion**

Conclusion

Key Takeaways

- ❶ **Kafka** decouples producers from consumers — 3-node KRaft cluster with 3 partitions per topic enables fault tolerance and parallel consumption
- ❷ **Flink** delivers sub-10 ms latency with exactly-once guarantees — 3 TaskManagers at parallelism 3 match Kafka partitions for maximum throughput
- ❸ **ELT pattern** keeps concerns separate: Flink loads raw events, dbt transforms inside Snowflake through landing → bronze → silver → gold layers
- ❹ **Observability** closes the loop — Prometheus & Grafana confirm the pipeline is healthy in real time

Future Directions

- Migrate to cloud — Confluent Cloud + Kinesis + Snowflake Snowpipe
- Add schema evolution with Confluent Schema Registry + Avro

- THE END -

Thank you for your attention

Contact:

Ngo Quang Hieu — Viettel Digital Talent 2026